502094 - Software Deployment, Operations And Maintenance

Module 5: Web Application Production Deployment Fundamentals

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Lecture Overview

- Understand the **end-to-end journey** from local development to production on AWS EC2
- Learn typical **deployment architectures** for Node.js and Spring Boot
- Practice **production-focused workflows**: building, configuring, and running apps
- See how **domains, HTTPS, and reliability** fit into real-world deployments
- Prepare for later modules on **load balancing, scaling, and containerization**

Agenda

- Local development foundations for web apps
- Preparing projects for **production deployment**
- Deployment architectures & patterns
- Provisioning and configuring **AWS EC2** servers
- Deploying **Node.js** and **Spring Boot** apps
- Database connectivity in production
- Verification, reliability, zero-downtime basics
- Domain & HTTPS configuration
- Comparing deployment targets
- Best practices and clean-up on AWS

Local Development Foundations

- Run and test applications **locally first** before touching servers
- Use **similar configs and environment variables** to what you expect in production
- Prepare **folders, logs, and build outputs** in a structured way
- Goal: your app should be **predictable and reproducible** on any machine

Running a Node.js Web App Locally (Concepts)

- Typical structure: `package.json`, `src/`, `public/`
- Dev server usually listens on **localhost:3000** or similar
- Use **scripts** in `package.json` for consistent commands
- Local run is where you **debug features** and **fix errors** before deployment

Running a Node.js Web App Locally

- `package.json`:

```
{
  "name": "devops-shop",
  "scripts": {
    "dev": "node src/server.js",
    "start": "NODE_ENV=production node src/server.js"
  }
}
```

- Local run:

```
npm install && npm run dev
```

- Access in browser: `http://localhost:3000`

Node.js Local Server Code Example

```
const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Hello from DevOps Shop (local dev)!');
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

- Uses **environment variable** `PORT` with fallback
- Same code can run **locally** and on **EC2**

Running a Spring Boot App Locally (Concepts)

- Typical structure: Maven or Gradle project
- Dev server listens on **localhost:8080** by default
- `application.properties` or `application.yml` contains config
- Use **Maven/Gradle wrapper** to avoid version mismatch

Running a Spring Boot App Locally (Example)

```
./mvnw spring-boot:run
# or
./gradlew bootRun
```

- Access in browser: `http://localhost:8080`
- Logs appear in terminal, including:
  - **Startup time**
  - **Active profiles**
  - **Port and context path**

Spring Boot Controller Example

```java
@RestController
public class HelloController {

    @GetMapping("/")
    public String index() {
        return "Hello from Spring Boot (local dev)!";
    }
}
```

- Once this works **locally**, we can think about:
  - Packaging into a **JAR**
  - Running on a **server** with proper configs

Dev vs. Production Configuration (Concepts)

- **Development**:
  - Verbose logs, in-memory DB, relaxed security
  - Hot reload, fast feedback
- **Production**:
  - External DB (e.g., AWS RDS)
  - Stricter security, metrics, and logging
  - Optimized builds and **read-only runtime environment**

Node.js: Using Environment Variables

- Use environment variables for:
  - **Database URLs**
  - **API keys and secrets**
  - **Port numbers and feature flags**

```
export NODE_ENV=development
export DB_URL="postgres://dev_user:dev_pass@localhost:5432/devops_shop"
npm run dev
```

- In code:

```
const dbUrl = process.env.DB_URL;
```

Spring Boot Profiles

- Spring Boot supports **profiles**: `dev`, `staging`, `prod`
- `application-dev.yml` vs `application-prod.yml`

```yaml
# application-dev.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/devops_shop_dev
    username: dev_user
    password: dev_pass
```

```yaml
# application-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://prod-db.internal:5432/devops_shop
    username: prod_app
    password: ${DB_PASSWORD}
```

Spring Boot Profiles

- Activate profile:

```
SPRING_PROFILES_ACTIVE=dev ./mvnw spring-boot:run
```

Preparing Folders and Logs Locally

- Prepare **consistent directories** used later in production:

```
mkdir -p logs
mkdir -p build
mkdir -p tmp/uploads
```

- Configure apps to write logs to `logs/app.log`
- This matches typical structure on a server, e.g. `/var/www/devops-shop/logs`

Preparing Projects for Production Deployment

- Structure repositories to separate **source, builds, and configs**
- Externalize environment-specific settings
- Build **optimized production artifacts**
- Ensure repository is **clean, reproducible, and secure**
- Push only **production-ready** code to GitHub

Repository Structure for Production

- Example layout:

```
devops-shop/
├─ src/
├─ config/
│  ├─ nginx/
│  └─ systemd/
├─ scripts/
├─ logs/        (ignored in Git)
├─ build/       (ignored in Git)
├─ .gitignore
└─ README.md
```

- `config/` holds **infra-related configs** used on the server
- `scripts/` holds deployment or utility scripts

Externalizing Configuration

- Do **not** hard-code secrets or URLs in code
- Read from environment variables or separate config files

```
const config = {
  port: process.env.PORT || 3000,
  dbUrl: process.env.DB_URL,
  jwtSecret: process.env.JWT_SECRET
};
```

- For Spring Boot, use **profiles** and **environment variables** as shown earlier
- This allows same binary to run in multiple environments

Node.js Production Builds

- Many Node.js web apps have a **build step**:

```
npm run build
```

- Common examples:
  - React/Next.js: bundles frontend assets
  - TypeScript: compiles to JavaScript
- In production:
  - Build in CI or on a build server
  - Deploy **built assets**, not TypeScript source
  - Use `npm ci` for reproducible installs

Spring Boot Fat JAR Packaging

- Build a **self-contained fat JAR**:

```
./mvnw clean package -DskipTests
```

- Output (example):

```
target/devops-shop-0.0.1-SNAPSHOT.jar
```

- Advantages:
  - Contains embedded Tomcat/Netty
  - Easy to run: `java -jar devops-shop-0.0.1-SNAPSHOT.jar`
  - Same JAR can be moved between servers

.gitignore for Production-Ready Repos

- Avoid committing **build outputs, logs, secrets**

```
# Node.js
node_modules/
build/
dist/
logs/
.env

# Java / Maven
target/
*.log
*.tmp
```

- Keep `.env.example` to document required variables
- This keeps the repo **clean and safe**

Pushing to GitHub Safely

- Typical workflow:

```
git status
git add .
git commit -m "Prepare production build pipeline"
git push origin main
```

- Before pushing:
  - Ensure **no secrets** in code, config, or `.env` files
  - Use tools like **git diff** to double-check
  - Principle: **never commit passwords or API keys**

Deployment Architecture & Patterns

- Understand how production environments are **structured**
- Separate **public-facing** and **internal** components
- Use **reverse proxies** to handle HTTPS and routing
- Prepare for future **scaling and load balancing**

Standard Production Architecture

- Typical pattern:

```
Client (Browser)
      ↓
Reverse Proxy (Caddy/Nginx)
      ↓
Application (Node.js / Spring Boot)
      ↓
Database (AWS RDS)
```

- Reverse proxy listens on **ports 80/443**
- Application listens on **internal port** (e.g., 3000, 8080)
- Database is **not exposed** to the internet

Why Not Expose 3000/8080 Publicly?

- Directly opening app port to the internet:
  - Increases **attack surface**
  - Makes HTTPS termination harder
  - Exposes **framework-specific errors** and stack traces
- Reverse proxy benefits:
  - Central place for **HTTPS, headers, logging**
  - Can route to **multiple apps**
  - Adds **rate limiting, caching, security** layers

Layer Separation

- **Network layer**:
  - Security groups, firewalls, load balancers
- **Application layer**:
  - Node.js / Spring Boot services
- **Data layer**:
  - Databases, caches, storage
- Each layer has its own **responsibilities** and **security rules**
- This separation helps with **debugging, scaling, and governance**

Public vs. Internal Ports

- Public-facing ports (e.g., **80, 443**) are reachable from the internet
- Internal ports (e.g., **3000, 8080, 5432**) should be limited to:
  - Same server
  - Private subnets
  - Specific security groups
- On AWS, use **Security Groups** to control which ports are accessible from where

Reverse Proxies: Role and Benefits

- Examples: **Caddy**, **Nginx**, **Traefik**
- Key tasks:
  - Terminate **TLS/HTTPS**
  - Route incoming requests to appropriate backend
  - Add common headers, logging, compression
- This makes backend services simpler and more **focused on business logic**

Single-Server vs. Multi-Server Deployments

- **Single-server**:
  - App + DB + reverse proxy on one EC2
  - Simple, good for **small projects or learning**
- **Multi-server**:
  - Separate app servers and DB server
  - Easier scaling and **fault isolation**
- Later modules: load balancers, auto-scaling groups, containers (Docker/Kubernetes)

Provisioning & Configuring AWS EC2 Production Servers

- Create **Ubuntu 22.04** EC2 instances for production
- Connect via **SSH** securely
- Install required runtimes: Node.js, Java, Git
- Configure firewall and basic hardening
- Prepare directory structure for apps

Creating an Ubuntu 22.04 EC2 Instance

- Key choices:
  - AMI: **Ubuntu Server 22.04 LTS**
  - Instance type: `t3.small` or `t3.medium` for small apps
  - Security Group:
    - Allow **22 (SSH)** from your IP
    - Allow **80/443** from anywhere (for HTTP/HTTPS)
- Attach a **key pair** for SSH authentication

Connecting via SSH

```
chmod 400 devops-key.pem
ssh -i devops-key.pem ubuntu@54.210.10.123
```

- Good practices:
  - Use **key-based authentication**, not passwords
  - Avoid logging in as **root**
  - Principle of **least privilege**: use normal user with `sudo`

Installing System Updates and Basic Tools

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl htop ufw
```

- `ufw` (Uncomplicated Firewall) to control ports
- Keep servers **patched** to reduce vulnerabilities
- Monitor CPU/memory with `htop` during load

Installing Node.js (LTS) on Ubuntu

```
# Using NodeSource (example for Node 20 LTS)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

- Always install **LTS** version for stability
- Use same major version in dev and prod for **consistency**

Installing Java JDK for Spring Boot

```
sudo apt install -y openjdk-21-jdk
java -version
```

- Ensure version matches what you use locally (e.g., **17 or 21**)
- Spring Boot logs will show which JVM is in use

Preparing Folder Structure on EC2

```
sudo mkdir -p /var/www/devops-shop
sudo mkdir -p /var/www/devops-shop/logs
sudo mkdir -p /opt/devops-shop
```

- Example conventions:
  - `/var/www` for web applications
  - `/opt` for supporting tools/scripts
- This structure makes it easier to **manage and back up** applications

Creating a Dedicated Linux User (Optional)

```
sudo adduser --system --group devops
sudo chown -R devops:devops /var/www/devops-shop
```

- Run apps as **non-root service accounts**
- Limits damage if an application is compromised
- Aligns with **least privilege** and security best practices

Deploying Node.js Applications to EC2

- Clone project from GitHub
- Install dependencies in **production mode**
- Use **PM2** to manage Node.js processes
- Configure automatic restart on reboot
- Inspect logs for debugging

Cloning the Repository on EC2

```
cd /var/www
sudo -u devops git \
  clone https://github.com/mvmanh/devops-shop-node.git devops-shop
cd devops-shop
```

- Use HTTPS or SSH depending on your setup
- Ensure proper **ownership** of files (e.g., user `devops`)

Installing Node.js Dependencies in Production Mode

```
# Recommended for CI / production
npm ci --only=production
```

- `npm ci`:
  - Uses `package-lock.json` for reproducible installs
  - Faster and more predictable than `npm install`
- Optional: run `npm run build` if your app needs a build step

Introducing PM2 for Process Management

- PM2 is a **process manager** for Node.js
- Features:
  - Keep app running forever
  - Restart on crash
  - Simple load balancing across CPU cores

```
sudo npm install -g pm2
pm2 start src/server.js --name devops-shop
pm2 status
```

PM2 Startup and Auto-Recovery

- Make PM2 restart apps after server reboot:

```
pm2 startup systemd
# Follow printed instructions, e.g.:
sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u devops --hp /home/devops

pm2 save
```

- PM2 will now restore process list from its **saved configuration**

PM2 Logs and Log Rotation

```
pm2 logs devops-shop
pm2 logs devops-shop --lines 100
```

- Logs help you:
  - Investigate **errors and slow requests**
  - Monitor **unexpected restarts**
- For long-term operation, integrate with:
  - `logrotate`
  - Centralized logging (ELK, Loki, etc.)

Deploying Spring Boot Applications to EC2

- Pull latest code from GitHub
- Build JAR locally or directly on the server
- Run using `java -jar` or **systemd** service
- Configure JVM memory and environment variables

Getting Spring Boot Code on EC2

```
cd /opt
sudo git clone https://github.com/mvmanh/devops-shop-spring.git
cd devops-shop-spring
```

- Option 1: build JAR locally and `scp` to server
- Option 2: build directly on EC2 (needs Maven/Gradle installed)

Building and Running the Spring Boot JAR

```
./mvnw clean package -DskipTests
ls target/
```

```
java \
  -Xms512m \
  -Xmx1024m \
  -Dspring.profiles.active=prod \
  -Dserver.port=8080 \
  -jar target/devops-shop-0.0.1-SNAPSHOT.jar
```

- Use appropriate **heap size** for your instance

Creating a systemd Service for Spring Boot

```
[Unit] # /etc/systemd/system/devops-shop.service
Description=DevOps Shop Spring Boot API
After=network.target
[Service]
User=devops
WorkingDirectory=/opt/devops-shop-spring
ExecStart=/usr/bin/java \
  -Xms512m \
  -Xmx1024m \
  -Dspring.profiles.active=prod \
  -Dserver.port=8080 \
  -jar target/devops-shop-0.0.1-SNAPSHOT.jar
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
```

Creating a systemd Service for Spring Boot

```
sudo systemctl daemon-reload
sudo systemctl enable devops-shop
sudo systemctl start devops-shop
sudo systemctl status devops-shop
```

Environment Variables for Spring Boot

```
sudo systemctl edit devops-shop
```

```
[Service]
Environment="DB_URL=jdbc:postgresql://devops-db.internal:5432/devops_shop"
Environment="DB_USERNAME=prod_app"
Environment="DB_PASSWORD=strong-password-here"
Environment="JWT_SECRET=change-me"
```

- Avoid hardcoding secrets in the **service file**
- Prefer reading from **environment** or secret stores

Database Connectivity in Production

- Connect apps to **managed databases** like AWS RDS
- Design **Security Groups** to limit DB access
- Configure **connection pools** for performance
- Handle credentials securely via environment variables
- Test and monitor **latency, failures, backups**

Using AWS RDS for Production

- Choose engine: **MySQL** or **PostgreSQL**
- Typical setup:
  - Private subnet RDS instance
  - Security Group allowing access from app servers only
- Example JDBC URL:

```
jdbc:postgresql://devops-db.internal:5432/devops_shop
```

Security Group Design for Databases

- RDS Security Group rules:
  - Inbound:
    - `5432` (PostgreSQL) from **app servers' Security Group**
  - No inbound from **0.0.0.0/0**
- This ensures:
  - Only your EC2 app instances can reach the DB
  - DB is invisible to the public internet

Connection Pools

- Spring Boot (HikariCP):

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      connection-timeout: 30000
```

Connection Pools

- Node.js (e.g., `pg` pool):

```
const { Pool } = require('pg');
const pool = new Pool({
  connectionString: process.env.DB_URL,
  max: 10,
  idleTimeoutMillis: 30000
});
```

- Connection pooling avoids opening a **new connection per request**

Handling DB Credentials via Environment Variables

```
export DB_URL="postgres://user:password@devops-db.internal:5432/devops_shop"
export DB_MAX_POOL=10
```

- Apps read from environment:
  - Prevents credentials from being pushed to Git
  - Allows easy rotation of passwords
- Consider using **AWS Systems Manager Parameter Store** or **Secrets Manager** in more advanced setups

Testing and Debugging DB Connectivity

```
# On EC2 app server
psql "$DB_URL" -c '\dt'
```

- Common issues:
  - Wrong hostname or port
  - Security Group denies access
  - Wrong credentials or database name
- Check:
  - Application logs
  - RDS logs / CloudWatch metrics

Verifying, Testing & Ensuring Reliability

- Verify deployments by testing **API endpoints**
- Detect issues with **permissions, ports, firewalls**
- Ensure apps **come back automatically** after reboot
- Use logs to troubleshoot and monitor health

Testing Application Endpoints

```
curl -i http://localhost:3000/health
curl -i http://localhost:8080/actuator/health
```

- Typical checks:
  - HTTP status code `200`
  - Response body indicates **UP**
  - Check via **reverse proxy URL** as well:

```
curl -i https://api.devops-shop.example/health
```

Detecting Port and Firewall Issues

- Check listening ports:

```
sudo ss -tulpn | grep -E '3000|8080|80|443'
```

- Check UFW rules:

```
sudo ufw status
```

- If app is running but unreachable:
  - Security Group may block traffic
  - Firewall may block port
  - Reverse proxy may be misconfigured

Ensuring Auto-Recovery (Node.js with PM2)

- Confirm PM2 startup is configured:

```
pm2 list
sudo reboot
```

- After reboot:
  - Check `pm2 list` again
  - Ensure `devops-shop` app is **online**
- This reduces downtime due to accidental reboots or crashes

Ensuring Auto-Recovery (Spring Boot with systemd)

- systemd service is set to:

```
Restart=always
RestartSec=5
```

- Test:

```
sudo systemctl status devops-shop
sudo systemctl stop devops-shop
sudo systemctl start devops-shop
```

- After reboot:

```
sudo systemctl status devops-shop
```

- Service should be **active (running)** automatically

Using Logs for Troubleshooting

- Node.js / PM2:

```
pm2 logs devops-shop
```

- Spring Boot (systemd + application logs):

```
journalctl -u devops-shop -n 100 -f
tail -f /var/www/devops-shop/logs/app.log
```

- Logs help you investigate:
  - Exceptions
  - Slow queries
  - Failed external calls

Zero-Downtime Deployment Basics

- Aim: keep service **available** while deploying new versions
- Understand differences between:
  - Restart
  - Reload
  - Redeploy
- On a single EC2 instance, options are limited but still useful

Restart vs. Reload vs. Redeploy

- **Restart**:
  - Stop and start the process
  - Short downtime
- **Reload**:
  - Replace code/config while keeping connections as graceful as possible
- **Redeploy**:
  - Deploy new version, point traffic to it (often with load balancer)
- True **zero-downtime** usually requires multiple instances + load balancer

PM2 Graceful Reload for Node.js

```
pm2 reload devops-shop
```

- PM2 will:
  - Start new process with updated code
  - Stop old process after existing requests finish
- Minimal downtime, but still limited if only **one instance**

systemd Restart Strategies for Spring Boot

- On single-server setups, you can minimize downtime by:
  - Deploying during **low-traffic periods**
  - Restarting the service quickly:

```
sudo systemctl restart devops-shop
```

- For advanced setups, combine systemd with **load balancers** and multiple instances

Blue-Green & Rolling Deployments (Conceptual)

- **Blue-Green**:
  - Two identical environments (blue and green)
  - Traffic switches entirely from blue to green when ready
- **Rolling**:
  - Update instances **one by one**
  - Load balancer keeps service available
- We will revisit these patterns when studying **load balancers and scaling**

Limits of Single-Instance Zero-Downtime

- With only one EC2 instance:
  - Hard to avoid **any** downtime during upgrades
  - Can reduce downtime with fast restarts and PM2 reloads
- Next step in maturity:
  - Multiple instances behind a **load balancer**
  - Rolling deploys with **health checks**

Domain & HTTPS Configuration

- Purchase a domain from a registrar
- Point DNS records to your EC2 **Elastic IP**
- Use **Caddy** for automatic HTTPS via Let's Encrypt
- Configure Caddy to reverse proxy Node.js / Spring Boot apps

Purchasing a Domain and Adding DNS Records

- Registrars: Namecheap, Cloudflare, AWS Route 53
- Example setup:
  - Domain: `devops-shop.xyz`
  - Elastic IP: `54.210.10.123`
- DNS A record:

```
api.devops-shop.xyz   A   54.210.10.123
```

- After DNS propagates, `api.devops-shop.xyz` will resolve to your EC2 instance

Installing Caddy on Ubuntu

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https

# Add Caddy GPG key
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/gpg.key | \
  sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg

# Add Caddy repository to sources list
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt | \
  sudo tee /etc/apt/sources.list.d/caddy-stable.list

# Update package list and install Caddy
sudo apt update
sudo apt install -y caddy
```

- Caddy will run as a systemd service listening on ports **80** and **443**

Caddyfile Configuration Example

- `api.devops-shop.xyz` is the domain

```
api.devops-shop.xyz {
  reverse_proxy localhost:3000
}
```

- Place in `/etc/caddy/Caddyfile`:

```
sudo nano /etc/caddy/Caddyfile
sudo systemctl reload caddy
```

- Caddy will:
  - Obtain and renew **TLS certificates** automatically
  - Forward HTTPS traffic to your Node.js app

Reverse Proxying Spring Boot with Caddy

```
api-spring.devops-shop.xyz {
  reverse_proxy localhost:8080
}
```

- With this setup:
  - Clients use `https://api-spring.devops-shop.xyz`
  - Caddy handles HTTPS
  - Spring Boot app continues listening on **localhost:8080**

Comparing Deployment Targets

- **Amazon EC2** vs. **Platform-as-a-Service (PaaS)**:
  - EC2: more control, more responsibility
  - PaaS (Render, Railway, Heroku, Netlify): less ops, more abstraction
- Choose based on **team size**, **budget**, and **requirements**

EC2 vs. PaaS: Trade-offs

- **EC2**:
  - Pros: Full control, flexible architecture, ideal for learning DevOps
  - Cons: Must manage OS, patches, monitoring, scaling
- **PaaS**:
  - Pros: Simpler deployment, automatic scaling options, built-in HTTPS
  - Cons: Less control, vendor lock-in, pricing can grow quickly

When to Choose EC2 vs. Managed Services

- Choose **EC2** when:
  - You need custom networking, security, or multi-service architecture
  - You want to learn real-world **DevOps skills**
- Choose **managed services** when:
  - You prioritize developer productivity over infra control
  - You have small teams or MVPs that need to go live quickly

Best Practices for Production Deployment on AWS

- Secure network configuration
- Proper secrets management
- Keep systems updated
- Monitor logs and disk usage
- Separate environments (dev, staging, prod)
- Avoid running as root

Locking Down Security Groups

- Principle: **allow only what is necessary**
- Example rules:
  - EC2 web server:
    - Allow SSH (22) from your IP only
    - Allow HTTP/HTTPS (80/443) from anywhere
  - DB (RDS):
    - Allow DB port only from app servers' Security Group
- Regularly review rules to avoid **overly permissive access**

Enforcing SSH Key Authentication

```
sudo nano /etc/ssh/sshd_config
```

- Ensure:

```
PasswordAuthentication no
PubkeyAuthentication yes
```

```
sudo systemctl reload sshd
```

- Use **unique key pairs** per admin
- Disallow root login directly when possible

Secrets Management

- Prefer environment variables over hardcoded secrets
- For larger systems, use:
  - **AWS Systems Manager Parameter Store**
  - **AWS Secrets Manager**
- Rotate secrets regularly
- Never store secrets in:
  - GitHub repositories
  - Docker images without encryption

Keeping Ubuntu Updated & Enabling Unattended Upgrades

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

- Enables automatic installation of **security updates**
- Reduces risk from known vulnerabilities

Log Rotation & Disk Monitoring

- Use `logrotate` for app logs:

```
sudo nano /etc/logrotate.d/devops-shop
```

```
/var/www/devops-shop/logs/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
    copytruncate
}
```

Log Rotation & Disk Monitoring

- Monitor disk usage:

```
df -h
```

- Prevents outages due to **full disks**

Separate Dev / Staging / Prod Environments

- Dev:
  - Frequent changes, internal-only access
- Staging:
  - Mirrors production as closely as possible
  - Used for pre-release testing
- Prod:
  - Stable, monitored, carefully controlled
- Each environment should have its own: Databases, Configurations, Access policies

Avoid Running as Root

- Run apps as **service-specific users** (e.g., `devops`)
- Use `sudo` only when needed for:
  - Installing packages
  - Editing system config
- This limits damage if app is compromised

Cleaning Up Resources after Deployment

- Important to avoid **unexpected AWS bills**
- Clean up unused:
  - EC2 instances
  - Elastic IPs
  - Security Groups, key pairs
  - EBS volumes and S3 buckets

Terminating EC2 and Releasing Elastic IPs

- Terminate EC2 instances that are no longer needed
- In AWS console:
  - Release **Elastic IPs** that are not associated
- Verify from your local machine:

```
ping api.devops-shop.xyz
```

- It should no longer resolve to the old IP after DNS updates

Cleaning Up Security Groups, Key Pairs, and Volumes

- Remove unused:
  - Security Groups
  - Key pairs
  - Detached EBS volumes
- Old resources can lead to:
  - **Security confusion**
  - **Unnecessary storage costs**

Removing S3 Buckets and Static Resources

- For static websites or assets in S3:
  - Delete objects or entire buckets no longer needed
- Always confirm that:
  - No application is still using those files
  - You have backups if required

Checking AWS Billing Dashboard

- Regularly review:
  - Current month charges
  - Cost breakdown per service
- Set **billing alerts** (e.g., email when cost exceeds threshold)
- Good habit for both students and professionals to avoid surprises

Summary & Next Steps

- You learned:
  - How to prepare Node.js and Spring Boot apps for production
  - Standard architecture: reverse proxy → app → database
  - How to deploy on AWS EC2 with PM2 and systemd
  - Basics of DB connectivity, reliability, and HTTPS
- Next modules:
  - **Load balancing, auto-scaling, containerization**
  - **Monitoring, logging, and observability**